Cursor Build Prompt — "Bhasha Mode" (Hindi/Indian Language Learning, Gaijin-Mode-style)

Paste this into Cursor's Composer/Agent mode as your initial project prompt. Rename "Bhasha Mode" to whatever you want.

PROJECT BRIEF

Build **Bhasha Mode**, a voice-first, immersive web app for learning Hindi (and extensible to other Indian languages: Tamil, Telugu, Marathi, Bengali). Instead of flashcard/quiz-style learning, users are dropped into scenario-based conversations with AI characters who live in a fictional Indian setting (a chai stall, a train station, a family wedding, a Delhi market). Users speak or type to characters, characters remember past conversations, and relationships deepen over time, unlocking backstory and harder content.

Core mechanics to replicate (conceptually, not copying any existing brand/UI):

1. **Voice-first conversation** — user speaks (STT) or types, character responds via TTS + text.
2. **Code-switching** — if the user doesn't know a word/phrase in Hindi, they can say it in English and instantly see/hear the Hindi translation, then continue the conversation without breaking flow.
3. **Scenario mode** — structured situations (ordering chai, asking directions, haggling at a market, meeting a friend's family) with a goal to complete.
4. **Free conversation mode** — open-ended chat with a character about their life, hobbies, family, opinions.
5. **Relationship/progression system** — XP or "closeness" score per character; hitting thresholds unlocks new topics, backstory, and harder scenarios.
6. **Memory** — characters recall facts the user told them in earlier sessions (stored per user-character pair).

TECH STACK

- **Frontend**: Next.js 14 (App Router) + TypeScript + Tailwind CSS + Framer Motion

- **Backend/DB/Auth**: Supabase (Postgres + Auth + Storage + Row Level Security)
- **LLM**: Google Gemini Flash (character dialogue generation, JSON scenario logic) — reuse same pattern as GlowAI's Sera+Gemini split if useful
- **STT**: Web Speech API for MVP (browser-native), fallback/upgrade path to Whisper API later
- **TTS**: Web Speech API voices for MVP; note for later upgrade to a proper Hindi-accurate TTS (e.g. ElevenLabs or Google Cloud TTS) since browser Hindi voices are inconsistent
- **Payments**: Razorpay (subscription, same as GlowAI)
- **Deployment**: Vercel

DATABASE SCHEMA (Supabase)

Create these tables with RLS enabled (user can only access their own rows):

```
-- users handled by Supabase Auth

characters (
  id uuid primary key,
  name text,
  bio text,
  personality_prompt text, -- system prompt
defining character voice/personality
```

```
  avatar_url text,
  location text, -- e.g. "Chai Stall",
"Delhi Metro"
  language text default 'hindi',
  difficulty_level text --
beginner/intermediate/advanced
)

scenarios (
  id uuid primary key,
  character_id uuid references characters,
  title text,
  goal_description text,
  system_prompt text, -- scenario-specific
instructions for the LLM
  difficulty_level text
)

user_character_progress (
  id uuid primary key,
  user_id uuid references auth.users,
  character_id uuid references characters,
  closeness_score int default 0,
  unlocked_topics text[],
  last_interacted_at timestamptz
)

conversation_messages (
  id uuid primary key,
  user_id uuid references auth.users,
  character_id uuid references characters,
  scenario_id uuid references scenarios
nullable,
  role text, -- 'user' | 'character'
```

```
  content_hindi text,
  content_english text, -- for code-switch
translation display
  audio_url text nullable,
  created_at timestamptz default now()
)

vocab_learned (
  id uuid primary key,
  user_id uuid references auth.users,
  word_hindi text,
  word_transliteration text,
  meaning_english text,
  learned_at timestamptz default now(),
  source_conversation_id uuid references
conversation_messages
)
```

PAGES / ROUTES

- `/` — Landing page explaining the concept, sample scenario preview, CTA to sign up
- `/auth` — Supabase auth (email + Google)
- `/dashboard` — List of characters/locations, progress per character, "continue conversation" cards
- `/character/[id]` — Character profile: bio, closeness score, unlocked topics, list of available scenarios

- `/talk/[characterId]` — The core conversation screen (see below)
- `/vocab` — Personal vocab list auto-collected from conversations, spaced-repetition style review
- `/pricing` — Free tier + paid tier (Razorpay)

THE CORE CONVERSATION SCREEN (`/talk/[characterId]`)

This is the heart of the app — spend the most build effort here.

Layout:

- Character avatar/illustration + subtle idle animation (Framer Motion)
- Chat transcript (Hindi text + optional transliteration + English translation toggle)
- Mic button (push-to-talk or tap-to-toggle) using Web Speech API for STT
- Text input fallback for typing
- A "Switch to English" inline button — if user is stuck, they type/say the phrase in English, get instant Hindi translation inline, then that phrase gets added to their vocab list automatically

Conversation flow:

1. On load, fetch character's `personality_prompt` + last N messages for context + current scenario's `system_prompt` (if in scenario mode)
2. Send to Gemini with a system prompt like:

```
You are {character.name}, a
{character.bio}. You are having a
conversation with a Hindi language
learner at {character.difficulty_level}
level.
Respond ONLY in Hindi (Devanagari
script), keep sentences short and
natural for a learner.
Stay in character. Reference past facts
the user has told you:
{memory_summary}.
If in scenario mode, gently guide
toward the goal:
{scenario.goal_description}.
Return JSON: { "hindi": "...",
"transliteration": "...", "english":
"..." }
```

3. Parse JSON response, display Hindi text, generate TTS audio, play it
4. On each exchange, run a lightweight extraction call (or heuristic) to pull 1-3 new vocab words into `vocab_learned`
5. Update `closeness_score` based on message count / scenario completion; check for unlock thresholds

Code-switch UX detail (this is the signature feature — nail this):

- Small "🌐 say it in English" button always visible near input
- User types/says English phrase → instant Gemini call returns Hindi translation + transliteration → shown as a translation chip they can tap to insert into their next message → conversation continues without losing context

---

SCENARIOS TO SEED (v1 content, adapt/expand)

1. Ordering chai and snacks at a roadside stall
2. Asking for directions at a train station
3. Bargaining for vegetables at a local market
4. Meeting a friend's family for the first time
5. Small talk with an auto-rickshaw driver
6. Casual college canteen conversation

Each needs 2-3 example exchanges hardcoded as few-shot examples in the scenario's `system_prompt` so the LLM stays on-script.

BUILD ORDER (tell Cursor to do this in phases, don't try all at once)

Phase 1 — MVP skeleton

- Next.js + Supabase auth + Tailwind setup
- Static character + scenario seed data
- Text-only conversation with one character, Gemini-powered, no voice yet

Phase 2 — Voice

- Add Web Speech API STT input
- Add TTS output
- Add code-switch translation flow

Phase 3 — Progression system

- closeness_score, unlocked_topics, vocab auto-collection
- Dashboard showing progress per character
- Vocab review page

Phase 4 — Monetization

- Razorpay subscription gate (e.g. free = 1 character + 10 msgs/day, paid = all characters unlimited)

INSTRUCTIONS TO CURSOR

Start with Phase 1 only. Scaffold the Next.js + Supabase project, set up the database schema above via Supabase migrations, seed 1 character + 1 scenario, and build the `/talk/[characterId]` page with text-only chat wired to Gemini (use an API route `/api/chat` that takes user message + character/scenario context and returns the JSON response). Ask me before moving to Phase 2.